

Python 应用开发基础

课程报告

通过 CSGO 战绩的数据用多种回归模型解决了 Rating 计算的问题

组号: 第 15 组

成员: 毛仙威, 邵克恩, 王博海, 史奕文

指导老师: 雷杰

2022 年 12 月 26 日

目录

1. 项目概要	3
2. 数据收集 --爬虫	4
3. 数据集的可视化展示	6
1. 首杀, 多杀, 残局的数据分布	6
2. 杀敌数与死亡数关系的折线图	7
3. 杀敌/死亡与 Rating 的关系图	7
4. 数据预处理	8
1. 划分数据集	8
2. 空值填充	8
3. 独热编码	9
4. 特征选择	10
5. 降维	11
6. 归一化	11
5. 机器学习	12
1. 线性 - LinearRegressor	12
2. 支持向量回归 - SupportVectorRegressor	13
3. K 邻近 - KNeighborsRegressor	14
4. 自适应提升算法 - AdaBoostRegressor	15
5. 决策树 - DecisionTreeRegressor	17
6. 随机森林 - RandomForestRegressor	18
7. 极端随机森林 - ExtraTreeRegressor	19
8. Bagging - BaggingRegressor	20
9. 梯度提升回归树 - GradientBoostingRegressionTree	22
10. 模型汇总比较	23
6. GUI	23
1) GUI 构成	23
2) 功能设计	24
7. 项目分工	25
8. 项目特色	25
9. 总结	26

1. 项目概要

5e 对战平台 rating 算法经历过一次更新。更新前，十分注重 KD，即杀敌数与死亡数的比率。只要人杀的多，死得少就能得到一个非常的 rating。但是这个算法的弊端逐渐显现出来，偏向 Baiter（喜欢卖队友，当老 6，注重保 KD）打法的玩家往往能获得高 rating。且与比赛的 rating 计算偏差较大。因此 5e 平台更新了这个算法。

新的算法则加大了首杀、多杀、场伤、rws 等数据对于 rating 的影响，即使 KD 不高，但是上述数据都比较高时，玩家的 rating 也能获得一个较高的值。然而具体算法并未公布给玩家们，这导致玩家总是觉得有时候算法不准确(有些玩家就会有明明我的很多数据都比另一个人的数据要高，但为什么 rating 会比他低的疑问)。

5e 平台新的 Rating 计算方法是不公开的，我们希望能够通过机器学习的方式解决 rating 计算的问题，让玩家能够自测 rating。



图 1: 5e 平台玩家战绩图

2. 数据收集 --爬虫

我们首先查看了 5e 平台官网的 Robots.txt 文件,发现官网禁止我们爬取社区模块的内容,因此玩家的战绩是被允许爬取的。

```
User-agent: *
Disallow: /forum
Disallow: /forum/
Disallow: /forum/*
```

图 2: 5e 网站的 Robots 协议

然而玩家的战绩网址是没有规律的并且网址没有绑定在玩家主页“查看详情”的按钮上,无法通过 html 页面的标签来获取。此外,它的后缀还是由时间以及一连串未知字符串组成,因此我们将所要爬取的网址链接都手动复制到 csv 文件中,然后在爬取代码中读取这些链接,直接进入到了战绩详情页。

1	src
2	https://arena.5eplay.com/data/match/g151-n-20221112155441343681424
3	https://arena.5eplay.com/data/match/g151-c-20221214214230866531082
4	https://arena.5eplay.com/data/match/g151-c-20221214204720561281199
5	https://arena.5eplay.com/data/match/g151-c-20221214195103259718892
6	https://arena.5eplay.com/data/match/g151-c-20221214190342557200947
7	https://arena.5eplay.com/data/match/g151-c-20221214180805455913591
8	https://arena.5eplay.com/data/match/g151-c-20221214171501457933326
9	https://arena.5eplay.com/data/match/g151-c-20221213233216858854592
10	https://arena.5eplay.com/data/match/g151-c-20221213225917960283708
11	https://arena.5eplay.com/data/match/g151-c-20221213212254562165373
12	https://arena.5eplay.com/data/match/g151-c-20221215173134656371184
13	https://arena.5eplay.com/data/match/g151-c-20221215154402595536969
14	https://arena.5eplay.com/data/match/g151-c-20221215022640760532369
15	https://arena.5eplay.com/data/match/g151-c-20221215014443552784504
16	https://arena.5eplay.com/data/match/g151-c-20221215005009556040326
17	https://arena.5eplay.com/data/match/g151-c-20221214233236358779313
18	https://arena.5eplay.com/data/match/g151-c-20221214222844660314790
19	https://arena.5eplay.com/data/match/g151-c-20221216164052961185521
20	https://arena.5eplay.com/data/match/g151-c-20221116030220131422516
21	https://arena.5eplay.com/data/match/g151-c-20221113235130584233439
22	https://arena.5eplay.com/data/match/g151-n-20221112222817563351275
23	https://arena.5eplay.com/data/match/g151-c-20221214211853461633855
24	https://arena.5eplay.com/data/match/g151-c-20221212224914559261746

图 3: src.csv (战绩网站) 示例

在我们撰写完爬虫代码后发现,当爬了大概 200 条数据时,我们的 ip 将会被封禁,为了解决这个问题,我们了解到可以使用代理 ip 的方法。我们可以生成一个具有很多个代理 ip 的网址链接,然后用代码爬取这些代理 ip 为一个列表。先测试代理 ip 的可用性,如果可用就进行爬取,如果不可用,就替换到下一个 ip。如此不断地用地代理 ip 达到我们爬取 1000 条以上数据的要求。

```

def get_ip_list(m_url):
    html=requests.get(url=m_url,
                      headers={'User-Agent':UserAgent().random}).text
    #按<br>分组
    t_arr=html.split('<br>')
    # 第一个特殊, 需要先按</script>\n分组
    t_0=t_arr[1].split('</script>\n')[1].strip
    ip_list=[]
    ip_list.append(t_0)
    # 第二个及后面直接遍历就行
    for i in range(2,len(t_arr)-1):
        ip_list.append(t_arr[i])
    return ip_list

```

图 4: 通过 url 来获取所有代理 ip

测试 ip 可用性:

timeout=5 表示发送请求后, 3 秒钟内返回响应, 否则就抛出异常;
time.sleep 可以减轻对方服务器的压力, 做一个“优雅”的爬虫;
若可用, 则返回 request 的实例, 若不可用则将其移除。

```

def test_ip(src,ip_list):
    for i in range(len(ip_list)-1,-1,-1):
        headers={
            'User-Agent':UserAgent().random
        }
        proxies={
            'http': f'http://{ip_list[i]}',
            'https': f'https://{ip_list[i]}'
        }
        try:
            time.sleep(1)
            r=requests.get(url=src,proxies=proxies,headers=headers,timeout=5)
            if r.status_code == 200:
                return r
        except Exception as e:
            ip_list.remove(ip_list[i])

```

图 5: 测试 ip 可用性的代码

最终我们爬到的数据

```
df = pd.read_csv("zhanji.csv", encoding="GBK")
df
```

	输赢	杀敌数	助攻数	死亡数	爆头率	首杀	多杀	残局	场伤	团队贡献	段位	rating
0	胜利	28	10	18	0.36	2	7	0	124.61	16.63	2735.0	1.46
1	胜利	26	1	19	0.23	9	8	0	87.36	12.74	3118.0	1.33
2	胜利	21	3	21	0.48	4	5	0	77.96	10.63	2713.0	0.98
3	胜利	17	4	16	0.47	1	2	0	71.89	9.58	3062.0	0.90
4	胜利	14	2	16	0.50	0	3	1	52.68	8.02	2442.0	0.83
...	...	...	...	...	...	...	...	...	...	...	...	...
1416	失败	20	2	16	0.50	2	7	1	82.41	8.13	2421.0	1.22
1417	失败	19	2	21	0.63	6	6	0	100.45	7.78	NaN	0.92
1418	失败	13	1	17	0.54	3	3	0	71.91	4.23	2014.0	0.81
1419	失败	9	3	19	0.22	2	1	0	72.45	4.04	NaN	0.53
1420	失败	7	0	17	0.29	0	0	0	33.45	2.30	2484.0	0.45

1421 rows × 12 columns

图 6：爬虫爬取到的数据

3. 数据集的可视化展示

1. 首杀，多杀，残局的数据分布

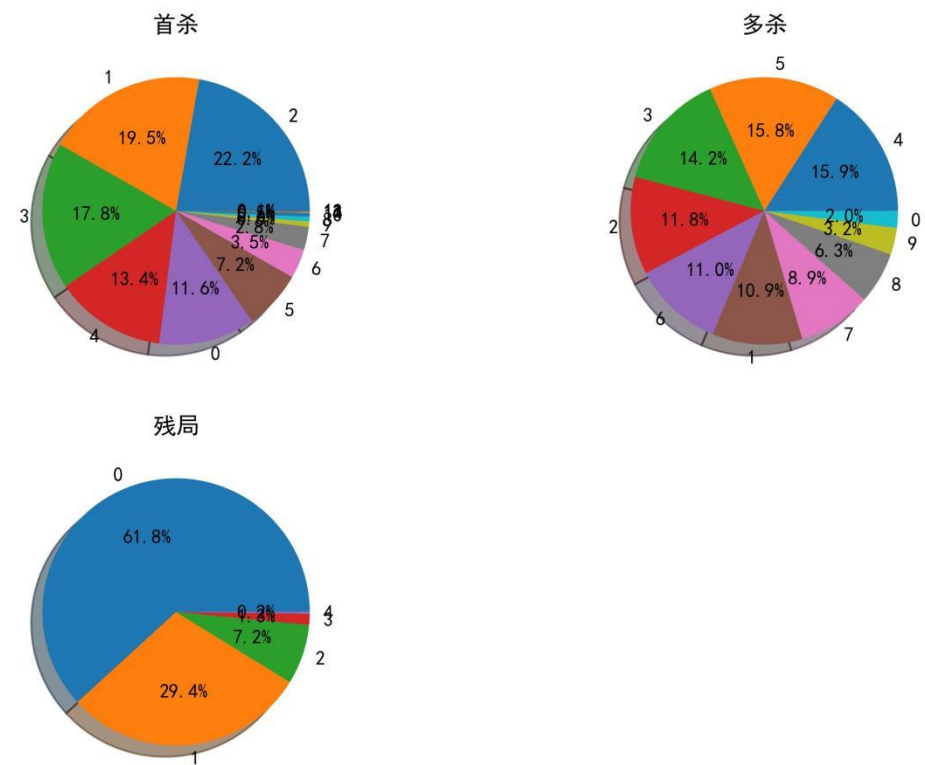


图 7：首杀多杀残局饼状图

从图中我们可以看出，首杀与多杀的数据分布是比较分散的，而残局的数据分布是比较集中的。

2. 杀敌数与死亡数关系的折线图

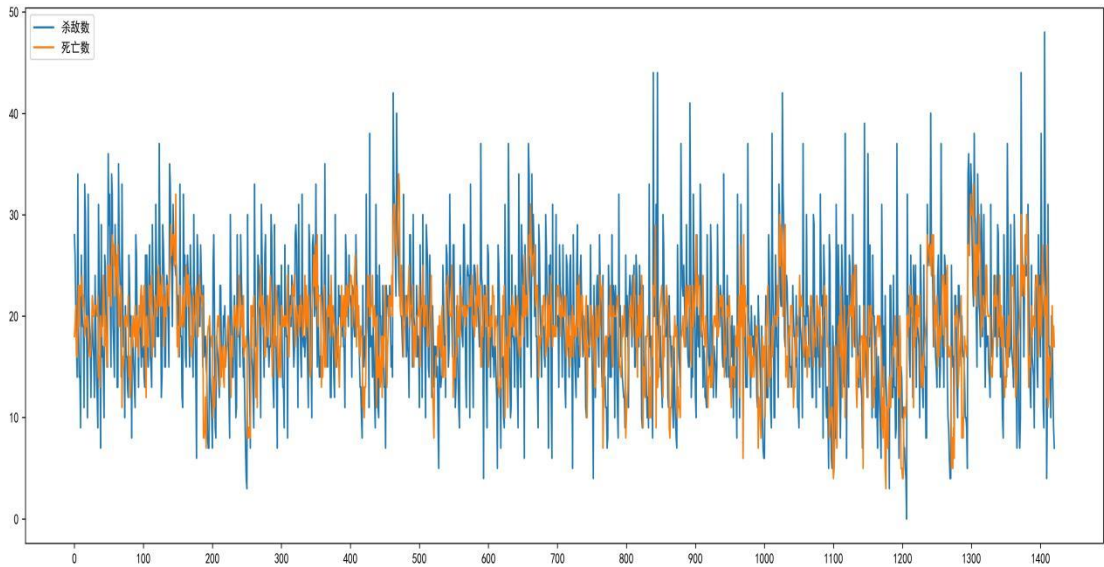


图 8：杀敌数与死亡数关系图

从这张折线图中我们可以看出，杀敌数的分布比较广泛，最大值与最小值的差距比较大；图中蓝色区域几乎是包围黄色区域的，可见死亡数的分布相较于杀敌数比较集中。

3. 杀敌/死亡与 Rating 的关系图

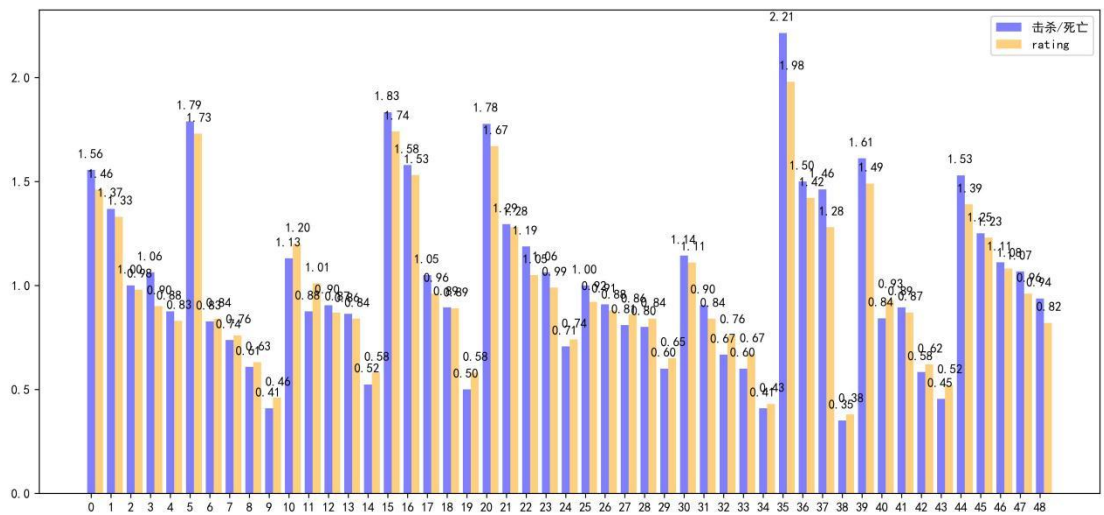


图 9：杀敌/死亡与 Rating 关系图

许多玩家都还认为Rating是由KD(击杀/死亡)来决定的。我们画出 49 个玩家的 Rating 和 KD 大小的条形图。可以看出，两者是存在一定正相关关系的，在大多数情况下是 KD 越高，Rating 越高，但这并不绝对，比如图中的 14 号和 19 号，他们的 Rating 相同，但是 KD 不

同；27 号和 28 号的 KD 相同，但是他们的 KD 不同。

4. 数据预处理

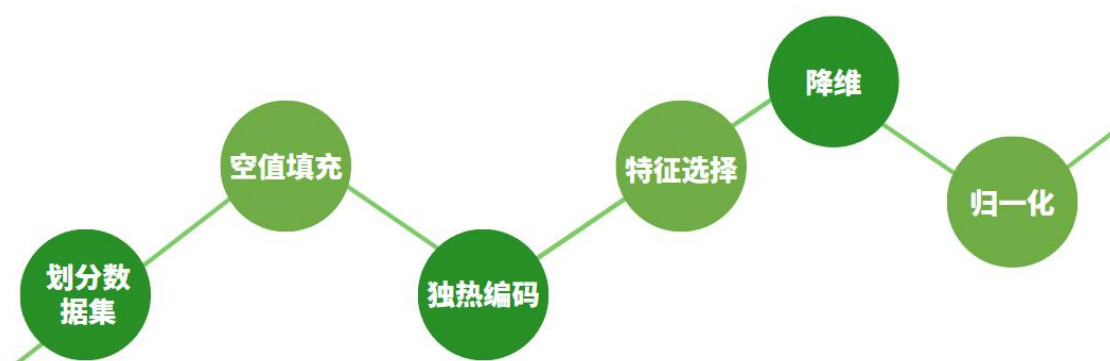


图 10：数据预处理顺序图

先划分数据集后在进行数据预处理，可以减小训练集数据对测试集数据的影响，提高模型的泛化能力。

1. 划分数数据集

```
x = df.iloc[:, :-1].copy()
y = df["rating"].copy()
x_train, x_test, y_train, y_test = train_test_split(x.copy(), y.copy(), test_size=0.3, shuffle=True)
x_train.index = np.arange(0, x_train.shape[0], 1)
x_test.index = np.arange(0, x_test.shape[0], 1)
```

图 11：划分数据集代码图

我们将数据集打乱后按照 7：3 的比例进行划分。

2. 空值填充

2.5 填充空值

```
# 对空值进行用平均值填充
x_train.fillna({'段位': x_train['段位'].mean()}, inplace=True)
x_test.fillna({'段位': x_test['段位'].mean()}, inplace=True)
x_train
```

图 12：空值填充代码图

由于部分玩家还处于定级赛阶段，没有段位，我们爬取后表格中为空值。由于对局中的玩家水平应当是相似段位，所以我们用平均值来进行填充。


```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1421 entries, 0 to 1420
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   输赢        1421 non-null   object
 1   杀敌数      1421 non-null   int64
 2   助攻数      1421 non-null   int64
 3   死亡数      1421 non-null   int64
 4   爆头率      1421 non-null   float64
 5   首杀        1421 non-null   int64
 6   多杀        1421 non-null   int64
 7   残局        1421 non-null   int64
 8   场伤        1421 non-null   float64
 9   团队贡献    1421 non-null   float64
10  段位        946 non-null    float64
11  rating      1421 non-null   float64
dtypes: float64(5), int64(6), object(1)
memory usage: 133.3+ KB
```

图 13: 原数据集存在空值

<pre><class 'pandas.core.frame.DataFrame'> Int64Index: 994 entries, 0 to 993 Data columns (total 11 columns): # Column Non-Null Count Dtype --- - 0 输赢 994 non-null object 1 杀敌数 994 non-null int64 2 助攻数 994 non-null int64 3 死亡数 994 non-null int64 4 爆头率 994 non-null float64 5 首杀 994 non-null int64 6 多杀 994 non-null int64 7 残局 994 non-null int64 8 场伤 994 non-null float64 9 团队贡献 994 non-null float64 10 段位 994 non-null float64 dtypes: float64(4), int64(6), object(1) memory usage: 133.2+ KB</pre>	<pre><class 'pandas.core.frame.DataFrame'> Int64Index: 427 entries, 0 to 426 Data columns (total 11 columns): # Column Non-Null Count Dtype --- - 0 输赢 427 non-null object 1 杀敌数 427 non-null int64 2 助攻数 427 non-null int64 3 死亡数 427 non-null int64 4 爆头率 427 non-null float64 5 首杀 427 non-null int64 6 多杀 427 non-null int64 7 残局 427 non-null int64 8 场伤 427 non-null float64 9 团队贡献 427 non-null float64 10 段位 427 non-null float64 dtypes: float64(4), int64(6), object(1) memory usage: 40.0+ KB</pre>
---	--

图 14: 训练集和测试集不存在空值

3. 独热编码

```
enc = OneHotEncoder(dtype=int, categories='auto')
oh = x_train.loc[:, "输赢"].values.reshape(-1, 1)
oh = enc.fit_transform(oh).toarray()
x_train = pd.concat((pd.DataFrame(oh, columns=enc.categories_[0]), x_train), axis=1)
x_train = x_train.drop("输赢", axis='columns')
joblib.dump(enc, 'model/OneHotEncoder.pkl')

oh = x_test.loc[:, "输赢"].values.reshape(-1, 1)
oh = enc.transform(oh).toarray()
x_test = pd.concat((pd.DataFrame(oh, columns=enc.categories_[0]), x_test), axis=1)
x_test = x_test.drop("输赢", axis='columns')
x_train
```

图 15: 独热编码代码图

由于输赢是中文字符，我们需要对其进行独热编码，我们将其分解为 3 个字段，分别为“失败”，“平局”，“胜利”。

同时，我们在数据预处理时还注意到了 `fit_transform` 和 `transform` 的区别，对训练集使用 `fit_transform`，对测试集使用 `transform`。因为机器学习假设是训练集和测试集的每个样本都是从同一分布中抽样得到的，所以在训练集中 `fit` 之后在测试集就不能再 `fit` 了，要不然二者可能不是同一分布模型，预测效果会不够准确。

Ps: 后续的数据预处理都遵守了这一原则

	失败	平局	胜利	杀敌数	助攻数	死亡数	爆头率	首杀	多杀	残局	场伤	团队贡献	段位
0	1	0	0	8	3	18	0.38	0	2	0	64.80	6.41	2524.000000
1	1	0	0	19	2	21	0.63	6	6	0	100.45	7.78	2640.097015
2	1	0	0	17	3	21	0.59	2	4	0	62.55	7.53	2719.000000
3	0	0	1	27	4	21	0.48	2	8	0	92.96	16.24	2640.097015
4	1	0	0	18	4	20	0.56	1	4	1	76.48	6.38	2521.000000
...	...	...	...	...	...	...	...	...	...	...	...	...	...
989	1	0	0	17	1	24	0.65	1	5	2	60.93	8.67	2652.000000
990	1	0	0	24	4	21	0.67	3	5	0	87.07	10.73	2640.097015
991	1	0	0	19	4	19	0.53	2	4	0	78.04	9.57	2640.097015
992	0	0	1	15	6	13	0.07	2	5	0	73.45	12.62	2386.000000
993	1	0	0	20	2	22	0.30	4	5	0	69.48	9.03	2548.000000

994 rows × 13 columns

图 16: 训练集_独热编码结果图

4. 特征选择

```
selector = VarianceThreshold(threshold=0)
x_train.iloc[:, :-1] = selector.fit_transform(x_train.iloc[:, :-1])
x_test.iloc[:, :-1] = selector.transform(x_test.iloc[:, :-1])
```

图 17: 特征选择代码图

在特征选择过程中，我们选择对方差为 0 的特征进行过滤，执行后，发现我们的特征中不存在此类特征。

5. 降维

```
pca = PCA(n_components=1)

pca_result = pca.fit_transform(x_train[["杀敌数", "场伤"]])
x_train.drop(["杀敌数", "场伤"], axis=1, inplace=True)
x_train = pd.concat((pd.DataFrame(pca_result, columns=["杀敌"]), x_train), axis=1)
joblib.dump(pca, 'model/PCA.pkl')

pca_result2 = pca.transform(x_test[["杀敌数", "场伤"]])
x_test.drop(["杀敌数", "场伤"], axis=1, inplace=True)
x_test = pd.concat((pd.DataFrame(pca_result2, columns=["杀敌"]), x_test), axis=1)
```

图 18: 降维代码图

经过分析，我们觉得杀敌数与场伤存在强相关，杀敌数高的用户往往场伤也会比较高，因此我们将这二者进行了降维的操作。

	杀敌	失败	平局	胜利	助攻数	死亡数	爆头率	首杀	多杀	残局	团队贡献	段位
0	-16.289451	1.0	0.0	0.0	3.0	18.0	0.38	0.0	2.0	0.0	6.41	2524.000000
1	20.979530	1.0	0.0	0.0	2.0	21.0	0.63	6.0	6.0	0.0	7.78	2640.097015
2	-16.212543	1.0	0.0	0.0	3.0	21.0	0.59	2.0	4.0	0.0	7.53	2719.000000
3	15.733032	0.0	0.0	1.0	4.0	21.0	0.48	2.0	8.0	0.0	16.24	2640.097015
4	-2.476361	1.0	0.0	0.0	4.0	20.0	0.56	1.0	4.0	1.0	6.38	2521.000000
...	...	...	...	...	...	...	...	...	...	...	...	...
989	-17.780862	1.0	0.0	0.0	1.0	24.0	0.65	1.0	5.0	2.0	8.67	2652.000000
990	9.279222	1.0	0.0	0.0	4.0	21.0	0.67	3.0	5.0	0.0	10.73	2640.097015
991	-0.715557	1.0	0.0	0.0	4.0	19.0	0.53	2.0	4.0	0.0	9.57	2640.097015
992	-6.161409	0.0	0.0	1.0	6.0	13.0	0.07	2.0	5.0	0.0	12.62	2386.000000
993	-8.751910	1.0	0.0	0.0	2.0	22.0	0.30	4.0	5.0	0.0	9.03	2548.000000

994 rows × 12 columns

图 19: 训练集_降维结果图

6. 归一化

```
for i in x_train.columns[:]:
    ss = StandardScaler()
    to_ss = x_train.loc[:, i].values.reshape(-1, 1)
    x_train.loc[:, i] = ss.fit_transform(to_ss)
    joblib.dump(ss, 'model/ss'+i+'.pkl')
    to_ss = x_test.loc[:, i].values.reshape(-1, 1)
    x_test.loc[:, i] = ss.transform(to_ss)
x_train
```

图 20: 数据归一化代码图

归一化的目的是使特征具有相同的度量尺度。若特征间具有不同的值范围时，因此梯度更新时，会来回震荡，经过较长的时间才能达到局部最优值或全局最优值。为了解决该模型问题，我们需要归一化数据，我们确保不同的特征具有相同的值范围，这样梯度下降可以很快的收敛。通常情况下归一化后训练出来的模型会比不归一化的模型效果更好。

	杀敌	失败	平局	胜利	助攻数	死亡数	爆头率	首杀	多杀	残局	团队贡献	段位
0	-0.695350	0.997990	-0.13191	-0.964415	-0.111774	-0.215462	-0.720075	-1.305752	-1.002507	-0.697734	-0.778274	-5.052954e-01
1	0.895555	0.997990	-0.13191	-0.964415	-0.633385	0.469514	0.913024	1.655277	0.754200	-0.697734	-0.487239	-1.979222e-15
2	-0.692067	0.997990	-0.13191	-0.964415	-0.111774	0.469514	0.651728	-0.318743	-0.124154	-0.697734	-0.540347	3.434138e-01
3	0.671598	-1.002014	-0.13191	1.036898	0.409838	0.469514	-0.066835	-0.318743	1.632554	-0.697734	1.309959	-1.979222e-15
4	-0.105709	0.997990	-0.13191	-0.964415	0.409838	0.241189	0.455756	-0.812247	-0.124154	0.694931	-0.784647	-5.183524e-01
...	...	...	...	...	...	...	...	...	...	...	...	...
989	-0.759014	0.997990	-0.13191	-0.964415	-1.154997	1.154489	1.043672	-0.812247	0.315023	2.087597	-0.298172	5.180601e-02
990	0.396103	0.997990	-0.13191	-0.964415	0.409838	0.469514	1.174320	0.174762	0.315023	-0.697734	0.139444	-1.979222e-15
991	-0.030545	0.997990	-0.13191	-0.964415	0.409838	0.012863	0.259784	-0.318743	-0.124154	-0.697734	-0.106980	-1.979222e-15
992	-0.263013	-1.002014	-0.13191	1.036898	1.453061	-1.357088	-2.745118	-0.318743	0.315023	-0.697734	0.540946	-1.105920e+00
993	-0.373594	0.997990	-0.13191	-0.964415	-0.633385	0.697839	-1.242667	0.668267	0.315023	-0.697734	-0.221695	-4.008389e-01

994 rows x 12 columns

图 21：训练集_归一化结果图

5. 机器学习

在这一小节中，我们对 sklearn 中的 9 个模型进行了测试，为的是选出一个效果最好的模型作为最终结果，并构建 GUI。

1. 线性 - LinearRegressor

1) 原理

线性回归通过构建线性模型对连续值进行预测。有主要以下 3 个模型优化步骤：
设立模型形式：

$$h(x) = \theta^T x$$

图 22:线性回归的模型形式图

设置损失函数：

损失函数(cost function 或 loss function)是用来衡量模型得到的预测值与真值之间的差距。是在通过迭代法求解模型参数时，优化问题的目标函数。

利用优化方法迭代更新得到模型：

模型参数的求解是一个无约束最优化问题，因此可以使用一些迭代法来求解；这里采用梯度下降法。迭代的规则如下：

$$\theta = \theta - \alpha * \frac{\partial L}{\partial \theta}$$

图 23:梯度下降法的迭代规则图

2) 结果

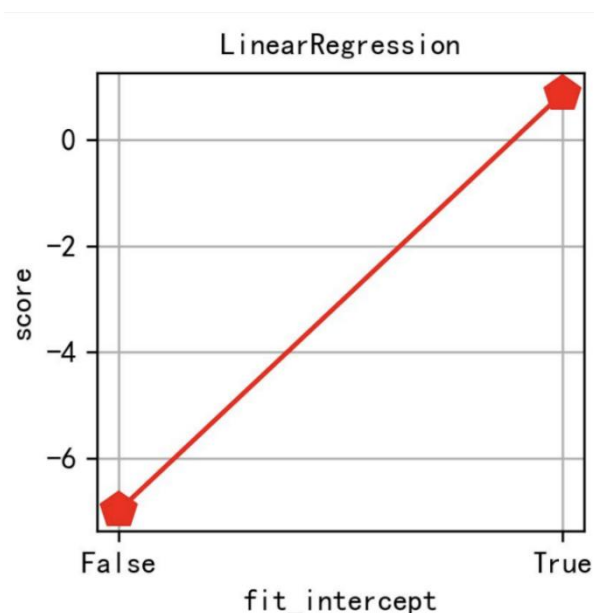


图 24: LinearRegressor 可视化结果图

由图可知, 根据调整参数 `fit_intercept`, 是否计算偏置可以看出, 计算偏置可以明显提高准确率。

```
{ 'score' : 0.8697051260528623, 'fit_intercept' : True }
```

图 25: LinearRegressor 最好结果图

2. 支持向量回归 - SupportVectorRegressor

1) 原理

支持向量回归 (support vector regression) 是支持向量机 (SVM) 的重要的应用分支。它与传统回归方法不同。区别在于:

传统回归方法当且仅当回归 $f(x)$ 完全等于 y 时才认为预测正确, 如线性回归中要计算其损失。而支持向量回归则认为只要计算值与实际值偏离程度不要太大, 即可以认为预测正确, 不用计算损失。

由此, 支持向量回归有以下优点:

1) 解决高维特征的分类问题和回归问题很有效, 在特征维度大于样本数时依然有很好的效果。

2) 仅仅使用一部分支持向量来做超平面的决策, 无需依赖全部数据。

3) 有大量的核函数可以使用, 从而可以很灵活的来解决各种非线性的分类回归问题。

4) 样本量不是海量数据的时候, 分类准确率高, 泛化能力强。

同时, 有以下缺点:

1) 如果特征维度远远大于样本数, 则 SVM 表现一般。

2) SVM 在样本量非常大, 核函数映射维度非常高时, 计算量过大, 不太适合使用。

3) 非线性问题的核函数的选择没有通用标准, 难以选择一个合适的核函数。

4) SVM 对缺失数据敏感。

2) 结果

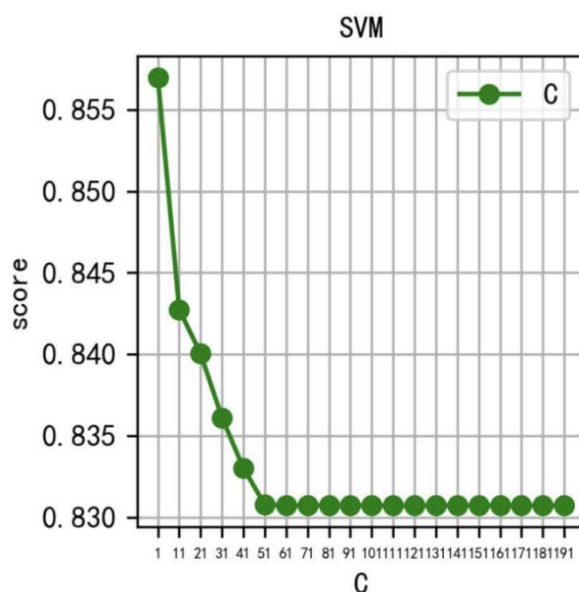


图 26: SupportVectorRegressor 可视化结果图

在 SVR 回归中，根据 C（错误项的惩罚因子），随着 C 的增大，即整个优化过程中对于总误差的关注程度变高，对于减小误差的要求变高时，准确率减小，并趋于稳定。

```
{ 'score': 0.8569745812061371, 'C': 1 }
```

图 27: SupportVectorRegressor 最好结果图

3. K 邻近 - KNeighborsRegressor

1) 原理

通过找出一个样本的 k 个最近邻居，将这些邻居的某个（些）属性的平均值赋给该样本，就可以得到该样本对应属性的值。

在 sklearn 中，KNN 回归中的 n_neighbors 就是 k 值。k 值如果比较小，会造成过拟合。如果 k 值比较大，无法将未知物体分类出来。

2) 结果

我们调整 k 值和邻居的权重的计算方法(weights)得到了如下结果:

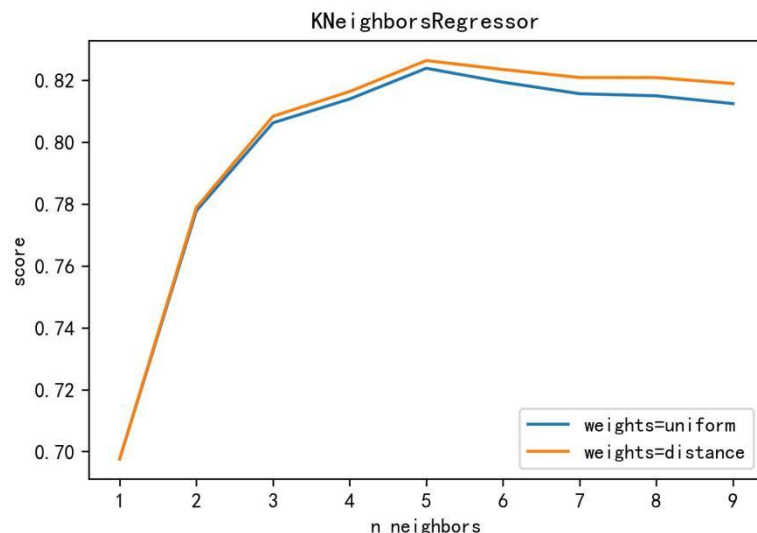


图 28: KNN 可视化结果图

从图中我们可以看出随着 `n_neighbors` 的增加, 准确率不断增加, 在 5 左右达到极值, 在之后有少许下降。对于权重计算方法, 当所有邻居节点的投票权重与距离成反比时, 效果比较好。准确率最大值和它的参数如下:

```
{'score': 0.8263507591853946, 'n_neighbors': 5, 'weights': 'distance'}
```

图 29: KNN 最好结果图

4. 自适应提升算法 - AdaBoostRegressor

1) 原理

Boosting 算法是集成算法中的一种, 同时也是一类算法的总称。这类算法通过训练多个弱分类器, 将它们组合成一个强分类器, 也就是我们俗话说的“三个臭皮匠, 顶个诸葛亮”。假设弱分类器为 $G_i(x)$, α_i 代表它在强分类器中的权重(如果弱分类器的分类效果好, 那么权重应该比较大, 如果弱分类器的分类效果一般, 权重应该降低), e_i 代表第 i 个分类器的分类错误率, 那么就可以得出强分类器 $f(x)$:

$$f(x) = \sum_{i=1}^n \alpha_i G_i(x) \quad \left(\alpha_i = \frac{1}{2} \log \frac{1-e_i}{e_i} \right)$$

图 30: Boost 公式图

AdaBoost 算法是通过改变样本的数据分布来实现的。AdaBoost 会判断每次训练的样本是否正确分类, 对于正确分类的样本, 降低它的权重, 对于被错误分类的样本, 增加它的权重。再基于上一次得到的分类准确率, 来确定这次训练样本中每个样本的权重。然后将修改过权重的新数据集传递给下一层的分类器进行训练。这样做的好处就是, 通过每一轮训练样

本的动态权重，可以让训练的焦点集中到难分类的样本上，最终得到的弱分类器的组合更容易得到更高的分类准确率。

我们可以用 D_{k+1} 代表第 $k+1$ 轮训练中，样本的权重集合，其中 $w_{\{k+1,1\}}$ 代表第 $k+1$ 轮中第一个样本的权重，以此类推 $w_{\{k+1,N\}}$ 代表第 $k+1$ 轮中第 N 个样本的权重，因此用公式表示为：

$$D_{k+1} = (w_{k+1,1}, w_{k+1,2}, \dots, w_{k+1,N})$$

图 31: AdaBoost 样本权重集合图

第 $k+1$ 轮中的样本权重，是根据该样本在第 k 轮的权重以及第 k 个分类器的准确率而定，具体的公式为：

$$w_{k+1,i} = \frac{w_{k,i}}{Z_k} \exp(-\alpha_k y_i G_k(x_i)), i = 1, 2, \dots, N$$

图 32: AdaBoost 样本权重计算图

2) 结果

将默认的 `base_estimator` 修改为 `DecisionTreeRegressor(max_depth=8)`，并通过调整 `n_estimators` 和损失函数，得到如下的结果图。

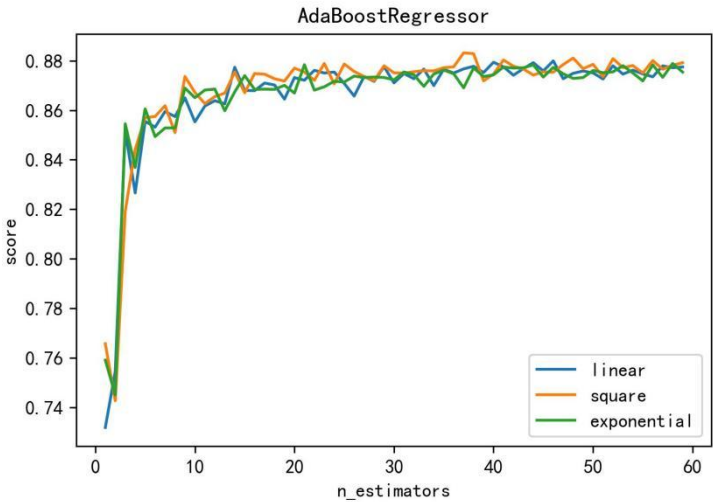


图 33: AdaBoost 可视化结果图

通过对图上的观察，我们可以得出，随着 `n_estimators` 的增加，准确率在不断上升，当达到 0.88 左右后趋于稳定，损失的计算方法对整体准确率的影响不大。

```
{ 'score': 0.8831869008130165, 'n_estimators': 37, 'loss': 'square' }
```

图 34: AdaBoost 最好结果图

5. 决策树 - DecisionTreeRegressor

1) 原理

构造就是生成一棵完整的决策树。简单来说，构造的过程就是选择什么属性作为节点的过程，那么在构造过程中，会存在三种节点：

根节点：就是树的最顶端，最开始的那个节点。

内部节点：就是树中间的那些节点，

叶节点：就是树最底部的节点，也就是决策结果。

剪枝就是给决策树瘦身，这一步想实现的目标就是，不需要太多的判断，同样可以得到不错的结果。之所以这么做，是为了防止“过拟合”（Overfitting）现象的发生。

信息熵：表示信息的不确定度

在信息论中，随机离散事件出现的概率存在着不确定性。为了衡量这种信息的不确定性，信息学之父香农引入了信息熵的概念，并给出了计算信息熵的数学公式：

$$Entropy(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$

图 35：信息熵计算公式图

$p(i|t)$ 代表了节点 t 为分类 i 的概率，其中 $\log_2$ 为取以 2 为底的对数。这里我们不是来介绍公式的，而是说存在一种度量，它能帮我们反映出来这个信息的不确定度。当不确定性越大时，它所包含的信息量也就越大，信息熵也就越高。

信息增益指的就是划分可以带来纯度的提高，信息熵的下降。它的计算公式，是父亲节点的信息熵减去所有子节点的信息熵。

$$Gain(D, a) = Entropy(D) - \sum_{i=1}^k \frac{|D_i|}{|D|} Entropy(D_i)$$

图 36：信息增益计算公式图

然后选择信息增益最大的部分作为节点。

2) 结果

`min_samples_leaf` 限定一个节点在分枝后的每个子节点都必须包含至少 `min_samples_leaf` 个训练样本，否则分枝就不会发生，或者，分枝会朝着满足每个子节点都包含 `min_samples_leaf` 个样本的方向去发生。一般搭配 `max_depth` 使用，在回归树中有神奇的效果，可以让模型变得更加平滑。

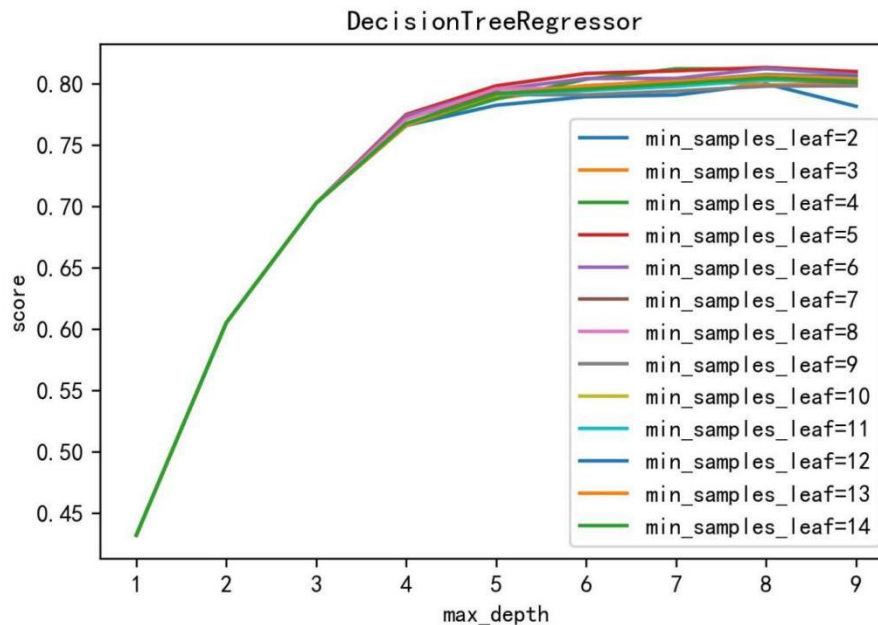


图 37: DecisionTreeRegressor 最好结果图

通过调整 max_depth 和 min_sample_leaf 的大小我们可以看出,随着 max_depth 的增大,无论 min_sample_leaf 取多大,准确率都在 max_depth = 6 左右达到最大值,之后的变化很缓慢。

```
{ 'score': 0.8131659050179272, 'max_depth': 8, 'min_samples_leaf': 5}
```

图 38: DecisionTreeRegressor 最好结果图

6. 随机森林 - RandomForestRegressor

1) 原理

随机森林回归算法 (Random Forest Regression) 是随机森林 (Random Forest) 的重要应用分支。随机森林回归模型通过随机抽取样本和特征,建立多棵相互不关联的决策树,通过并行的方式获得预测结果。每棵决策树都能通过抽取的样本和特征得出一个预测结果,通过综合所有树的结果取平均值,得到整个森林的回归预测结果。

随机森林回归的过程如下:

步骤 1: 对训练集进行有放回随机抽样以获得 p 个样本形成训练集的一个子集作为新的训练集;

步骤 2: 当每个样本有 K 个属性时,在决策树的每个节点需要分裂时,随机从这 K 个属性中选取 k 个属性,满足条件 $k < K$ 。然后从这 k 个属性中采用方差来选择一个属性作为该节点的分裂属性;

步骤 3: 决策树形成过程中每个节点都要按照步骤 2 来分裂。一直到不能够再分裂为止,利用该子集训练一棵决策树,并且不对这个决策树进行剪枝;

步骤 4: 按照步骤 1~3 建立大量的决策树,直至训练出 m 个决策树;

步骤 5: 把测试样本给每棵决策树进行回归预测,统计所有决策树对同一样本的预测结果,所有的结果的平均值作为最终预测值。

2) 结果

根据各参数的作用、对程序运行设备的性能等因素的考量，对随机森林回归模型的调整主要着手于 `n_estimators` 以及 `max_samples`。

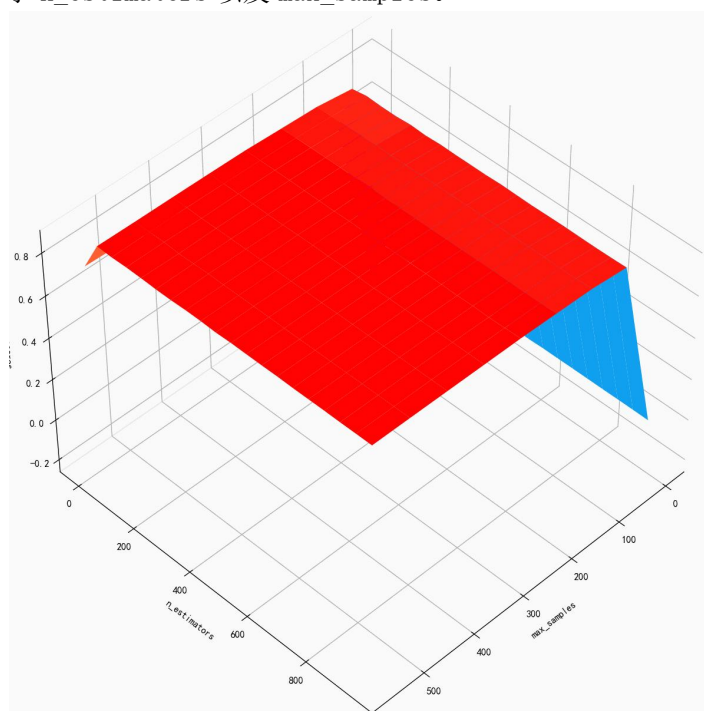


图 39: RandomForestRegressor 可视化结果图

由图 33 可以看出，`n_estimators` 与 `max_samples` 都与正确率 `score` 正相关，且变化趋势都是先快后慢，最终趋于平缓。然而，由于两个参数的增大都会影响程序的运行速度，应当取得的最优模型不应该是尽可能大的 `n_estimators` 与 `max_samples`。考虑到变化趋势逐渐趋于平稳，最优模型的获取采用了以下规则：若当前模型的正确率大于目前最优模型超过 0.02，则选取为目前最优模型。

最终取得的最优模型 `n_estimators = 51`, `max_samples = 561`。

```
{ 'score': 0.8788482998237146, 'n_estimators': 51, 'max_samples': 561 }
```

图 40: RandomForestRegressor 最好结果图

7. 极端随机森林 - ExtraTreeRegressor

1) 原理

极端随机森林同样是一种多棵决策树集成的分类器，与随机森林分类器比较，主要有两点不同：

对于每个决策树的训练集，RF 采用的是随机采样 `bootstrap` 来选择采样集作为每个决策树的训练集，而 `extra tree` 一般不采用随机采样，即每个决策树采用原始训练集。RandomForest 应用的是 Bagging 模型，ExtraTree 使用的所有的样本，只是特征是随机选取的，因为分裂是随机的，所以在某种程度上比随机森林得到的结果更加好。

在选定了划分特征后，RF 的决策树会基于信息增益，基尼系数，均方差之类的原则，

选择一个最优的特征值划分点，这和传统的决策树相同。但是 Extra tree 比较的激进，会随机的选择一个特征值来划分决策树。

从第二点可以看出，由于随机选择了特征值的划分点位，而不是最优点位，这样会导致生成的决策树的规模一般会大于 RF 所生成的决策树。也就是说，模型的方差相对于 RF 进一步减少，但是 bias 相对于 RF 进一步增大。在某些时候，Extra tree 的泛化能力比 RF 更好。一般情况下，极端随机森林分类器在分类精度和训练时间等方面都要优于随机森林分类器。

2) 结果

经过对各个参数进行改变计算正确率并用 matplotlib 画图发现，只有两个参数是对预测正确率有较为明显的影响，即 `min_samples_split` 和 `min_samples_leaf`。

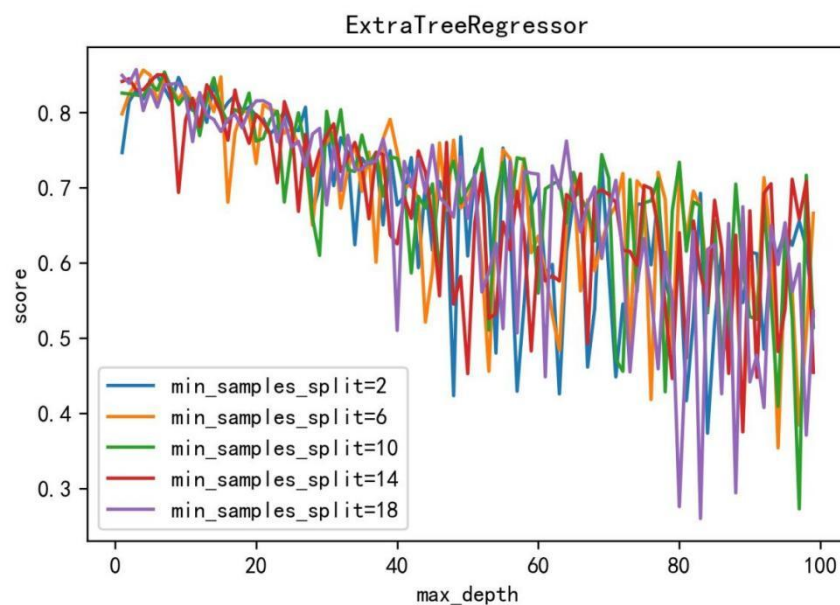


图 41: ExtraTreeRegressor 可视化结果图

由图可知，当 `min_samples_split` 和 `min_samples_leaf` 增大时，起初正确率会有一个较小的升高，但在后面随着两个参数值的增大，正确率逐渐下降。可以看到，当 `min_samples_split` 为 2 和 `min_samples_leaf` 为 7 时，得到的正确率是最高的。

```
{ 'score': 0.8379910677015228, 'min_samples_leaf': 7, 'min_samples_split': 2 }
```

图 42: ExtraTreeRegressor 最好结果图

8. Bagging - BaggingRegressor

1) 原理

Bagging 是通过结合几个模型降低泛化误差的技术。主要想法是分别训练几个不同的模型，然后让所有模型表决测试样例的输出。这是机器学习中常规策略的一个例子，被称为模型平均（`model averaging`）。采用这种策略的技术被称为集成方法。

模型平均（`model averaging`）奏效的原因是不同的模型通常不会在测试集上产生完

全相同的误差。模型平均是一个减少泛化误差的非常强大可靠的方法。在作为科学论文算法的基准时，它通常是不鼓励使用的，因为任何机器学习算法都可以从模型平均中大幅获益（以增加计算和存储为代价）。

算法步骤：给定一个大小为 n 的训练集 D ，Bagging 算法从中均匀、有放回地（即使用自助抽样法）选出 m 个大小为 n' 的子集 D_i ，作为新的训练集。在这 m 个训练集上使用分类、回归等算法，则可得到 m 个模型，再通过取平均值、取多数票等方法，即可得到 Bagging 的结果。

Bagging 回归便是 Bagging 算法与回归算法的结合，即在 Bagging 算法得到的训练集上使用回归算法。

2) 结果

处于与随机森林回归相同的理由，模型调整时同样选择了 `n_estimators` 和 `max_samples` 作为调整的参数。调试结果图如下：

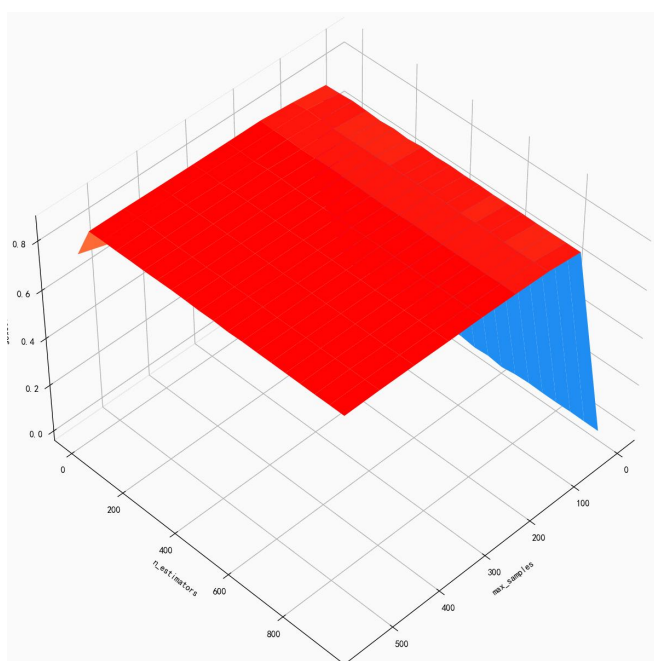


图 43: BaggingRegressor 可视化结果图

可以看出，`score` 的变化趋势和随机森林回归基本相同，推测是由于两个模型的同名参数在两个模型里的作用基本一致导致的，也因此采用了与随机森林回归相同的最优模型筛选机制。得到的最优模型如下图：

```
{ 'score': 0.8712997361673125, 'n_estimators': 51, 'max_samples': 211 }
```

图 44: BaggingRegressor 最好结果图

9. 梯度提升回归树 -

GradientBoostingRegressionTree

3) 原理

GBRT 算法是集成学习 Boosting 家族的成员，使用了前向分布算法，但是弱学习器限制了只能使用 CART 回归树模型。前向分布算法的思想是基于当前模型和拟合函数来选择合适的决策树函数，从而最小化损失函数。

GBRT 主要有以下两部分组成：

回归树 (Regression Tree (RT))：回归树是决策树类别之一，用来预测实际值。GBRT 算法是一种迭代的回归树算法，由多棵回归树组成，所有回归树的结论累加起来为最终结果。

梯度提升 (Gradient Boosting (GB))：梯度提升通过迭代多棵树来共同决策最终结果。每一棵树都是学习之前的树的结论和残差。

有以下 4 种参数用于调参：n_estimators (迭代次数)、objective (目标)、max_depth (最大深度)、random_state (随机状态)

4) 结果

经过对各个参数进行改变计算正确率并用 matplotlib 画图发现，只有 n_estimators (迭代次数) 这一参数是对预测正确率有较为明显的影响。

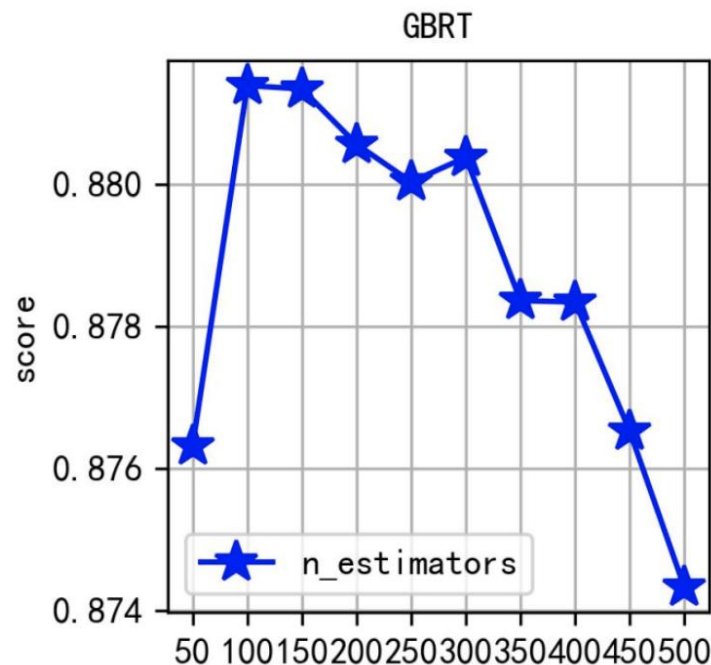


图 45: GBRT 可视化结果图

在 GBRT 中，根据 n_estimators，随着 n_estimators 的增大，准确率先变高，当 n_estimators=100 时，准确率最高。而后，准确率降低

```
{ 'score': 0.8813851305950129, 'n_estimators': 100 }
```

图 46: GBRT 最好结果图

10. 模型汇总比较

我们将上述 9 个模型的最好值画成柱状图进行直观的比较。

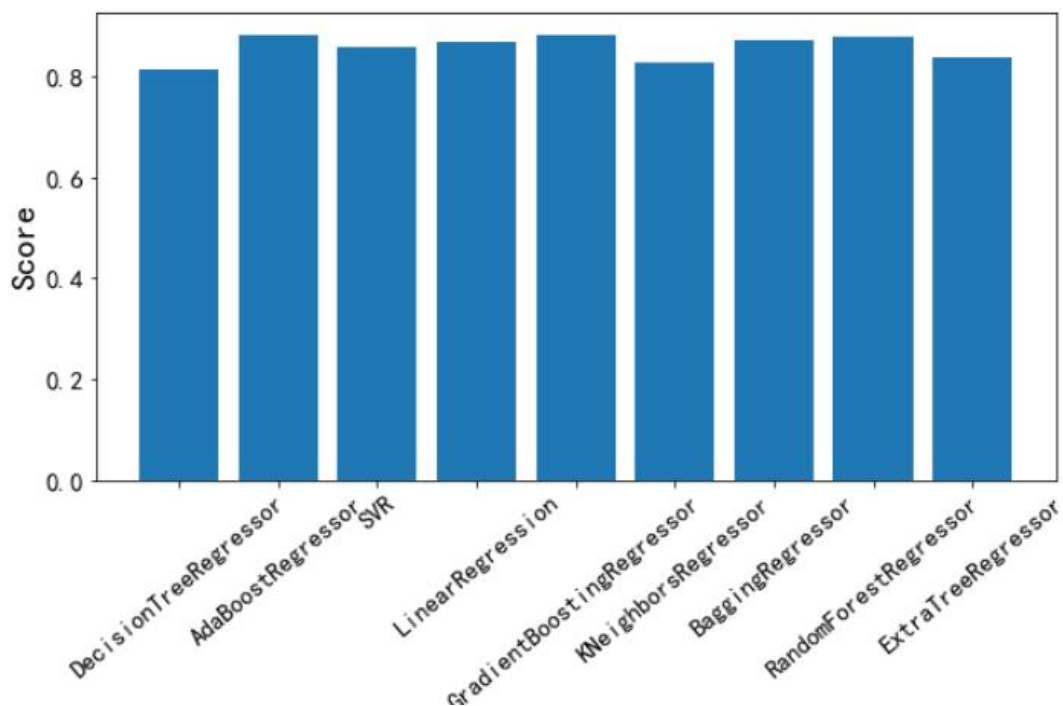


图 47: 模型比较图图

得到的最好结果如下:

模型名 AdaBoostRegressor
参数与Score { 'score': 0.8831869008130165, 'n_estimators': 37, 'loss': 'square' }

图 48: 最优模型结果图

6. GUI

1) GUI 构成

采用框架: Tkinter

实例: 1 个 Tkinter 窗口, 10 个 Entry 输入框, 13 个 Label 标签, 1 个 ComboBox 下拉框, 1 个 Button 按钮

布局: 表格布局



The image shows a window titled "Rating 预测器" (Rating Predictor). It contains a dropdown menu for "输赢" (Win/Loss) set to "平局" (Draw). Below it are ten input fields for various statistics: "杀敌数" (Kills), "助攻数" (Assists), "死亡数" (Deaths), "爆头率" (Headshot Rate), "首杀" (First Kill), "多杀" (Multi Kill), "残局" (Clutch), "场伤" (Damage), "团队贡献" (Team Contribution), and "段位" (Rank). A "提交" (Submit) button is located below the input fields. At the bottom, the "Rating" label is visible.

图 49: GUI (无输入)

2) 功能设计

在填入各项数据以及选择输赢后，点击提交，下方 Rating 处就会显示预测的值，效果如下：



The image shows the same "Rating 预测器" window, but now with data entered in the input fields. The "输赢" dropdown is set to "胜利" (Victory). The input fields contain the following values: "杀敌数" (26), "助攻数" (1), "死亡数" (19), "爆头率" (0.23), "首杀" (9), "多杀" (8), "残局" (0), "场伤" (87.36), "团队贡献" (12.74), and "段位" (3118). The "提交" button is still present. Below the input fields, the "Rating" label is followed by the predicted value "[1.33]" in green text.

图 50: GUI (有输入)

功能相关代码如下：


```

def pretreat(df):
    enc = joblib.load('model/OneHotEncoder.pkl')
    df.fillna({'段位': 0}, inplace=True)
    oh = df.loc[:, "输赢"].values.reshape(-1, 1)
    oh = enc.transform(oh).toarray()
    df = pd.concat((pd.DataFrame(oh, columns=enc.categories_[0]), df), axis=1)
    df = df.drop("输赢", axis='columns')
    pca = joblib.load('model/PCA.pkl')
    pca_result = pca.transform(df[["杀敌数", "场伤"]])
    df.drop(["杀敌数", "场伤"], axis=1, inplace=True)
    df = pd.concat((pd.DataFrame(pca_result, columns=["杀敌"]), df), axis=1)
    for i in df.columns[:]:
        ss = joblib.load('model/ss' + i + '.pkl')
        to_ss = df.loc[:, i].values.reshape(-1, 1)
        df.loc[:, i] = ss.transform(to_ss)
    return df

def callback():
    global dstr
    win = value.get()
    kill = K_entry.get()
    assist = A_entry.get()
    dead = D_entry.get()
    head_burst_rate = H_entry.get()
    first = F_entry.get()
    continuous = C_entry.get()
    endgame = E_entry.get()
    damage = damage_entry.get()
    team = T_entry.get()
    rank = rank_entry.get()
    model = joblib.load('model/' + 'AdaBoostRegressor' + '.pkl') # model = joblib.load('model/' + max_list2[max_index] + '.pkl')
    ar = np.array([win, kill, assist, dead, head_burst_rate, first, continuous, endgame, damage, team, rank])
    x = pd.DataFrame(index=range(0, 1),
                     columns=["输赢", "杀敌数", "助攻数", "死亡数", "爆头率", "首杀", "多杀", "残局", "场伤", "团队贡献", "段位"])
    x.loc[0] = ar
    x = pretreat(x)
    pm = model.predict(x)
    dstr.set(pm)

```

图 51: GUI 代码

上下两个方法分别为预处理方法与按钮事件。

点击提交按钮后，按钮事件方法会获取各个输入框的参数并整合为 array 形式，然后用预处理方法进行预处理，再输入到之前运行得到的最优模型，就可以得到预测值，并通过动态字符串显示在 Label 中。

7. 项目分工

在分工上，我们团队根据团队成员能力以及兴趣，将项目分成 3 个部分进行再分工，分别是数据收集与处理、机器学习模型的建立以及后期的展现。团队成员各有着重，有条不紊地推进项目的发展。

以下是团队成员的具体分工：

毛仙威(组长): 数据分析与数据预处理; KNN, 决策树, Adaboost 回归模型

邵克恩: GUI; 随机森林回归、Bagging 回归模型

王博海: 数据爬取; 极端森林回归模型; 答辩

史奕文: 线性回归、SVM、GBRT 回归模型

PPT 由 4 人各自负责编写所做工作的部分，使用了在线协作网站，可以直接统一风格。

文档由 4 人各自负责编写所做工作的部分，最后由毛仙威修改并整合。

8. 项目特色

1. 选题具有创意性，同时也是小组成员的兴趣所在

项目由当下最热门的 FPS 游戏入手，探究 Rating 的计算问题。同时，我们团队在可行性和课程贴合度上进行对选题的多维度评估，最终选择“通过 CSGO 战绩的数据用多种回归模型解决 Rating 计算问题”作为我们的项目主题。

2. 项目贴合课程内容

与课堂中 Python 数据可视化、爬虫、机器学习、线性模型和树模型等内容关系紧密，将学到的理论知识付诸于实践进行再一轮的巩固强化。

3. 注重于各个回归模型的建立和分析比较

在回归模型建立的同时，我们将深度剖析每个模型的优缺点，并进行比较各个场景对回归模型的使用方法，并在文档中详细展现。

4. 以目标（计算 Rating 准确性）为导向

在项目过程中，每个小组成员目标明确，以准确计算出 Rating 值为导向，不断优化模型，更正错误，总结经验。

5. 小组成员间分工明确、配合紧密

科学对项目进行拆解并根据小组成员的能力和兴趣分配任务，再加上紧密的交流，使项目高效率高质量的得以完成。

9. 总结

1. 课程设计总结

目前我们每次跑完代码，预测的准确率都在 0.88~0.91 之间，在之后，我们会继续优化实验中的各个流程，以求达到更好的效果。

通过本次课程设计，我们小组成员对从数据获取到机器学习模型的训练与测试，这一整套完整的机器学习流程已经非常熟练。在实验过程中，我们比较深刻地体验并感受到了整个流程中每一个步骤的重要性。例如课程设计主题的选择，我们小组开会讨论了很久，有许多非常好的想法，但是在第一步数据获取就直接被否定了，原因是找不到符合我们想法的数据集或适合爬取的网站。

2. 课程总结

在这门课程的学习过程中，我们对机器学习的了解也达到了入门的水平，我们学习了很多方面的知识，包括大量数据获取的方式——爬虫；数据的可视化；数据的预处理；重点分析了机器学习中的各个经典算法，同时也介绍了深度学习中的 Pytorch 框架以及计算机视觉方面的知识。最后讨论了 AI 前沿研究目前面临的挑战。让我们感受到 AI 的世界真是一个缤纷多彩的世界，我们生活中处处都在接触着 AI。